Embedded Control for Robotics

Lecture # 09 Control Design

Book: Embedded Digital Control with Microcontrollers
Chapter # 8

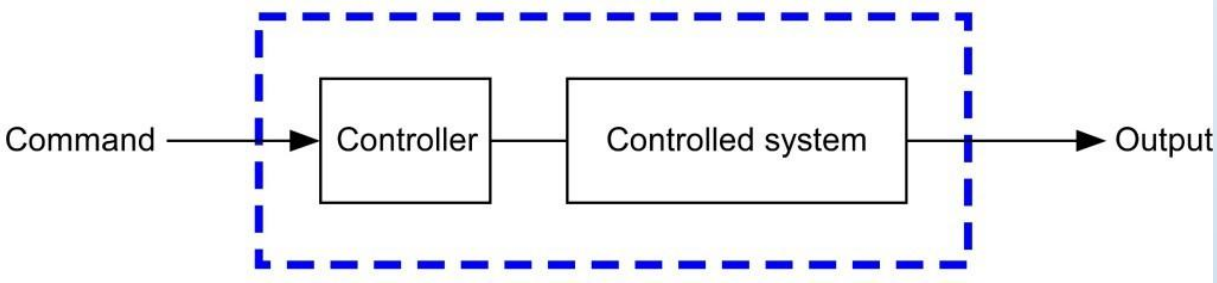
Dr. Suleman Khan
CS, NUCES-FAST, Islamabad

Overview Today's Lecture

- Control System
- On-off Control
- PID Controller
- PID Tuning
- Control Design given Specification

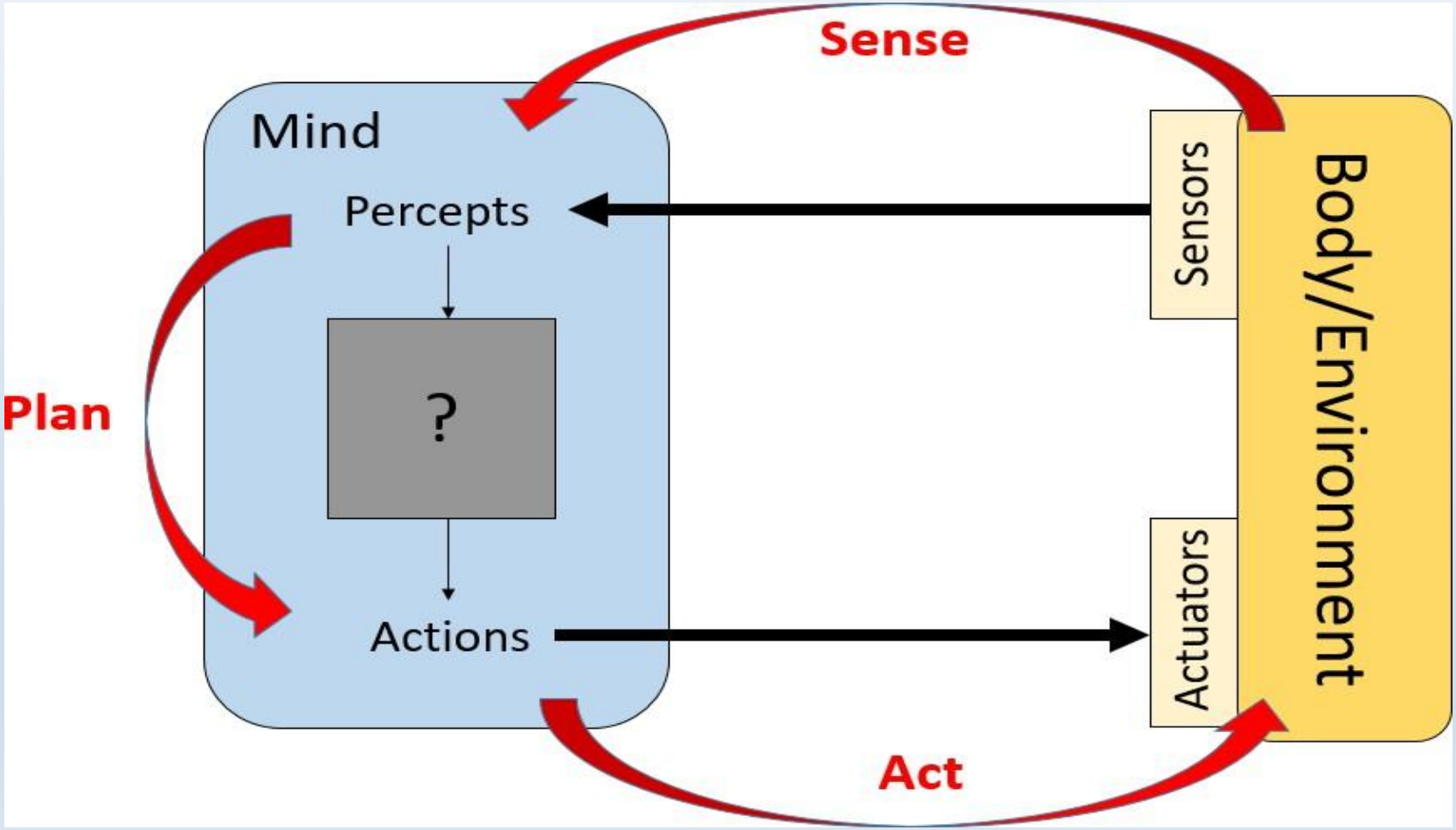
Control System

- A control system should contain a controller to perform various control functions on the system to get the desired outputs.



- A complete control system should be composed of inputs, the controller, the system or a process, and the system outputs.

Big Picture


Sense
Mind
Percepts
?
Plan
Actions
Sensors
Actuators
Body/Environment
Act

Control Design

8.1 Consider the DC motor with transfer function

$$G(s) = \frac{165702.47}{s^2 + 390.2s + 15701}$$

a. design P, PI, and PID controllers for the DC motor using the Cohen–Coon method. Form a separate closed-loop system for each controller separately.
b. Plot the unit step response for the closed-loop systems.
c. Compare the closed-loop systems in terms of time domain performance criteria.
d. Obtain Bode plots for the closed-loop systems.
e. Compare the closed-loop systems in terms of frequency domain performance criteria.

Design: Step 1 System Analysis

$$G(s) = \frac{165702.47}{s^2 + 390.2s + 15701}$$

1-Poles of system: let's find the system's characteristics by analyzing the denominator to find poles:

$$s^2 + 390.2s + 15701 = 0$$

s1 = -45.56,
s2 = -344.64

This is a second-order system that can be represented as:

$$G(s) = \frac{K}{(\tau_1 s + 1)(\tau_2 s + 1)}$$

Where:

tau_1 = 1/45.56 = 0.02195 sec
tau_2 = 1/344.64 = 0.00290 sec
K = 165702.47/15701} = 10.55$

Using unit Step Response

- Simulate open-loop unit step, find inflection point (max slope), draw tangent at that point. The tangent intercept with time axis gives L (dead time);
- The distance from that intercept to the intercept with steady state gives T (time constant).
- Process gain: K=10.5522827411
- Dead time: L=0.00174293017 sL=0.00174293017 s (≈1.74 ms)
- Time constant: T=0.02987094967
- sT=0.02987094967 s (≈29.87 ms)
- (So the plant is fast and has very small apparent dead time.)

Step 2: Cohen-Coon Method Parameters

For Cohen-Coon method, we need:

tau = \tau_1 = 0.02195$ sec (larger time constant)

theta = \tau_2 = 0.00290$ sec (smaller time constant = time delay approximation)

K_p =

The DC (steady-state) gain of the plant (for unit step input) is

$$K_{\text{process}} = G(0) = \frac{165702.47}{15701} \approx 10.5523.$$

Ratio: = theta\tau = 0.00290/0.02195 = 0.132$

Phython code


```
!pip install control matplotlib numpy pandas
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import control as ctrl


# Plant
num = [165702.47]
den = [1.0, 390.2, 15701.0]
G = ctrl.TransferFunction(num, den)


# transfer function
print(G)
t = np.linspace(0, 0.2, 20001)
t, y = ctrl.step_response(G, T=t)

plt.plot(t, y)
plt.show()
# Find parameters for Cohen-Coon method
# Steady-state value
K = y[-1]

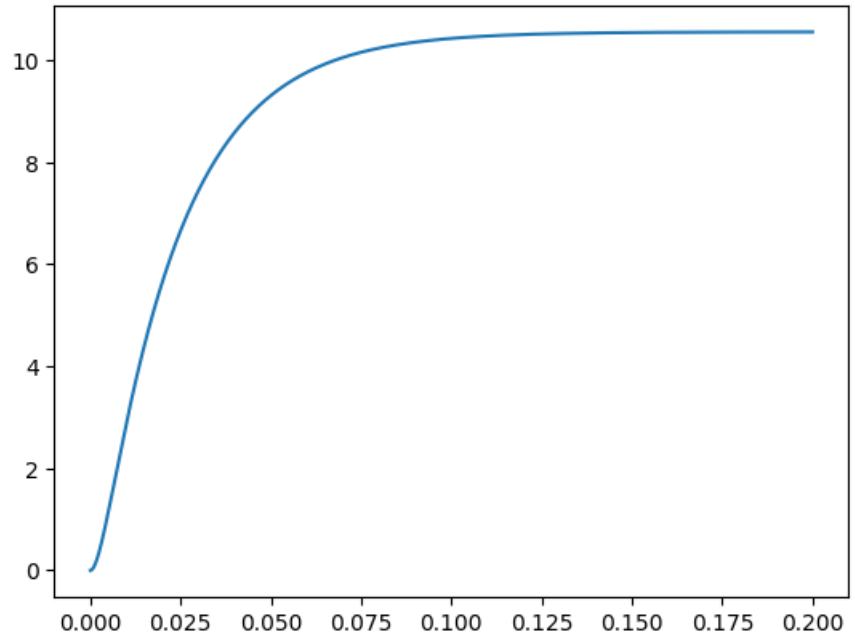
# Find time when response reaches 28.3% and 63.2% of final value
y_283 = 0.283 * K
y_632 = 0.632 * K

# Find corresponding times
idx_283 = np.argmax(y >= y_283)
idx_632 = np.argmax(y >= y_632)


tau = t[idx_632] - t[idx_283]
theta = t[idx_283]

print(f"System parameters:")
print(f"K = {K:.4f}")
print(f"τ = {tau:.4f}")
print(f"θ = {theta:.4f}")
print(f"θ/τ = {theta/tau:.4f}")
```

```
        1.657e+05
------------------------
s^2 + 390.2 s + 1.57e+04
```




```
System parameters:
K = 10.5523
τ = 0.0148
θ = 0.0103
θ/τ = 0.6960
```

Step: Controller

- **P**:

$$K_c = \frac{\tau}{K\theta}\left(1 + \frac{\theta}{3\tau}\right)$$

- **PI**:

$$K_c = \frac{\tau}{K\theta}\left(0.9 + \frac{\theta}{12\tau}\right), \qquad T_i = \theta\frac{30 + 3(\theta/\tau)}{9 + 20(\theta/\tau)}$$

- **PID** (standard PID with K_c, T_i, T_d):

$$K_c = \frac{\tau}{K\theta}\left(\frac{4}{3} + \frac{\theta}{4\tau}\right), \qquad T_i = \theta\frac{32 + 6(\theta/\tau)}{13 + 8(\theta/\tau)}, \qquad T_d = \theta\frac{4}{11 + 2(\theta/\tau)}.$$

Step 3: Controller Design using Cohen-Coon

3.1 P Controller

Cohen-Coon formula for P controller:

$$K_c = \frac{1}{K} \cdot \frac{\tau}{\theta} \left(1 + \frac{\theta}{3\tau}\right)$$

$$K_c = \frac{1}{10.55} \cdot \frac{0.02195}{0.00290} \left(1 + \frac{0.00290}{3 \times 0.02195}\right)$$

$$K_c = 0.0948 \times 7.569 \times (1 + 0.0441)$$

$$K_c = 0.717 \times 1.0441 = 0.748$$

P Controller: $C(s) = 0.748$

Continue…

3.2 PI Controller

Cohen-Coon formulas:

$$K_c = \frac{1}{K} \cdot \frac{\tau}{\theta}\left(0.9 + \frac{\theta}{12\tau}\right)$$

$$\tau_I = \theta \cdot \frac{30+3\frac{\theta}{\tau}}{9+20\frac{\theta}{\tau}}$$

$$K_c = \frac{1}{10.55} \cdot \frac{0.02195}{0.00290}\left(0.9 + \frac{0.00290}{12\times 0.02195}\right)$$

$$K_c = 0.0948 \times 7.569 \times (0.9 + 0.0110)$$

$$K_c = 0.717 \times 0.911 = 0.653$$

$$\tau_I = 0.00290 \cdot \frac{30+3\times 0.132}{9+20\times 0.132}$$

$$\tau_I = 0.00290 \cdot \frac{30.396}{11.64} = 0.00290 \times 2.611 = 0.00757$$

PI Controller: $C(s) = 0.653 \left(1 + \frac{1}{0.00757 s}\right) = 0.653 + \frac{86.26}{s}$

Continue… PID

Cohen-Coon formulas:

$$K_c = \frac{1}{K} \cdot \frac{\tau}{\theta} \left(\frac{4}{3} + \frac{\theta}{4\tau} \right)$$
$$\tau_I = \theta \cdot \frac{32 + 6\frac{\theta}{\tau}}{13 + 8\frac{\theta}{\tau}}$$
$$\tau_D = \theta \cdot \frac{4}{11 + 2\frac{\theta}{\tau}}$$

$$K_c = \frac{1}{10.55} \cdot \frac{0.02195}{0.00290} \left(1.333 + \frac{0.00290}{4 \times 0.02195} \right)$$
$$K_c = 0.0948 \times 7.569 \times (1.333 + 0.0330)$$
$$K_c = 0.717 \times 1.366 = 0.979$$

$$\tau_I = 0.00290 \cdot \frac{32 + 6 \times 0.132}{13 + 8 \times 0.132}$$
$$\tau_I = 0.00290 \cdot \frac{32.792}{14.056} = 0.00290 \times 2.333 = 0.00677$$

$$\tau_D = 0.00290 \cdot \frac{4}{11 + 2 \times 0.132}$$
$$\tau_D = 0.00290 \cdot \frac{4}{11.264} = 0.00290 \times 0.355 = 0.00103$$

PID Controller: $C(s) = 0.979 \left(1 + \frac{1}{0.00677 s} + 0.00103 s\right)$

Phyton code & output

```
# Cohen-Coon tuning calculations
K_process = K
tau_process = tau
theta_process = theta
ratio = theta_process / tau_process

# P Controller
Kc_P = (1/K_process) * (tau_process/theta_process) * (1 + theta_process/(3*tau_process))

# PI Controller
Kc_PI = (1/K_process) * (0.9*tau_process/theta_process) * (1 + theta_process/(12*tau_process))
tau_I = theta_process * (30 + 3*ratio) / (9 + 20*ratio)

# PID Controller
Kc_PID = (1/K_process) * (4*tau_process/(3*theta_process)) * (1 + theta_process/(4*tau_process))
tau_I_PID = theta_process * (32 + 6*ratio) / (13 + 8*ratio)
tau_D = theta_process * 4 / (11 + 2*ratio)

print(f"\nCohen-Coon Tuning Parameters:")
print(f"P Controller: Kc = {Kc_P:.4f}")
print(f"PI Controller: Kc = {Kc_PI:.4f}, τI = {tau_I:.4f}")
print(f"PID Controller: Kc = {Kc_PID:.4f}, τI = {tau_I_PID:.4f}, τD = {tau_D:.4f}")
```

```
Cohen-Coon Tuning Parameters:
P Controller: Kc = 0.1677
PI Controller: Kc = 0.1296, τI = 0.0144
PID Controller: Kc = 0.2131, τI = 0.0200, τD = 0.0033
```

Step 4: Closed-Loop Transfer Functions

4.1 P Controller System

$$T_P(s) = \frac{0.748 \times \frac{165702.47}{s^2+390.2s+15701}}{1+0.748 \times \frac{165702.47}{s^2+390.2s+15701}}$$

$$T_P(s) = \frac{123946.25}{s^2+390.2s+15701+123946.25}$$

$$T_P(s) = \frac{123946.25}{s^2+390.2s+139647.25}$$

Continue...

4.2 PI Controller System

$$C(s)G(s) = \frac{0.653(s+132.1)}{s} \times \frac{165702.47}{s^2+390.2s+15701}$$

$$T_{PI}(s) = \frac{108204.71(s+132.1)}{s(s^2+390.2s+15701)+108204.71(s+132.1)}$$

4.3 PID Controller System

$$C(s)G(s) = \frac{0.979(0.00103s^2+s+147.71)}{s} \times \frac{165702.47}{s^2+390.2s+15701}$$

$$T_{PID}(s) = \frac{227.4993\,s^2 + 362757.0688\,s + 86641196.8715}{s^3 + 617.6993\,s^2 + 378458.0688\,s + 86641196.8715}$$

Step 5: Phyton Code for Simulation


```
s = ctrl.TransferFunction([1,0],[1])
C_P = ctrl.TransferFunction([Kc_P], [1])

# PI Controller
C_PI_num = [Kc_PI * tau_I, Kc_PI]
C_PI_den = [tau_I, 0]
C_PI = ctrl.TransferFunction(C_PI_num, C_PI_den)

# PID Controller
C_PID_num = [Kc_PID * tau_D * tau_I_PID, Kc_PID * tau_I_PID, Kc_PID]
C_PID_den = [tau_I_PID, 0]
C_PID = ctrl.TransferFunction(C_PID_num, C_PID_den)


G_cl_P = ctrl.feedback(C_P*G, 1)
G_cl_PI = ctrl.feedback(C_PI*G, 1)
G_cl_PID = ctrl.feedback(C_PID*G, 1)

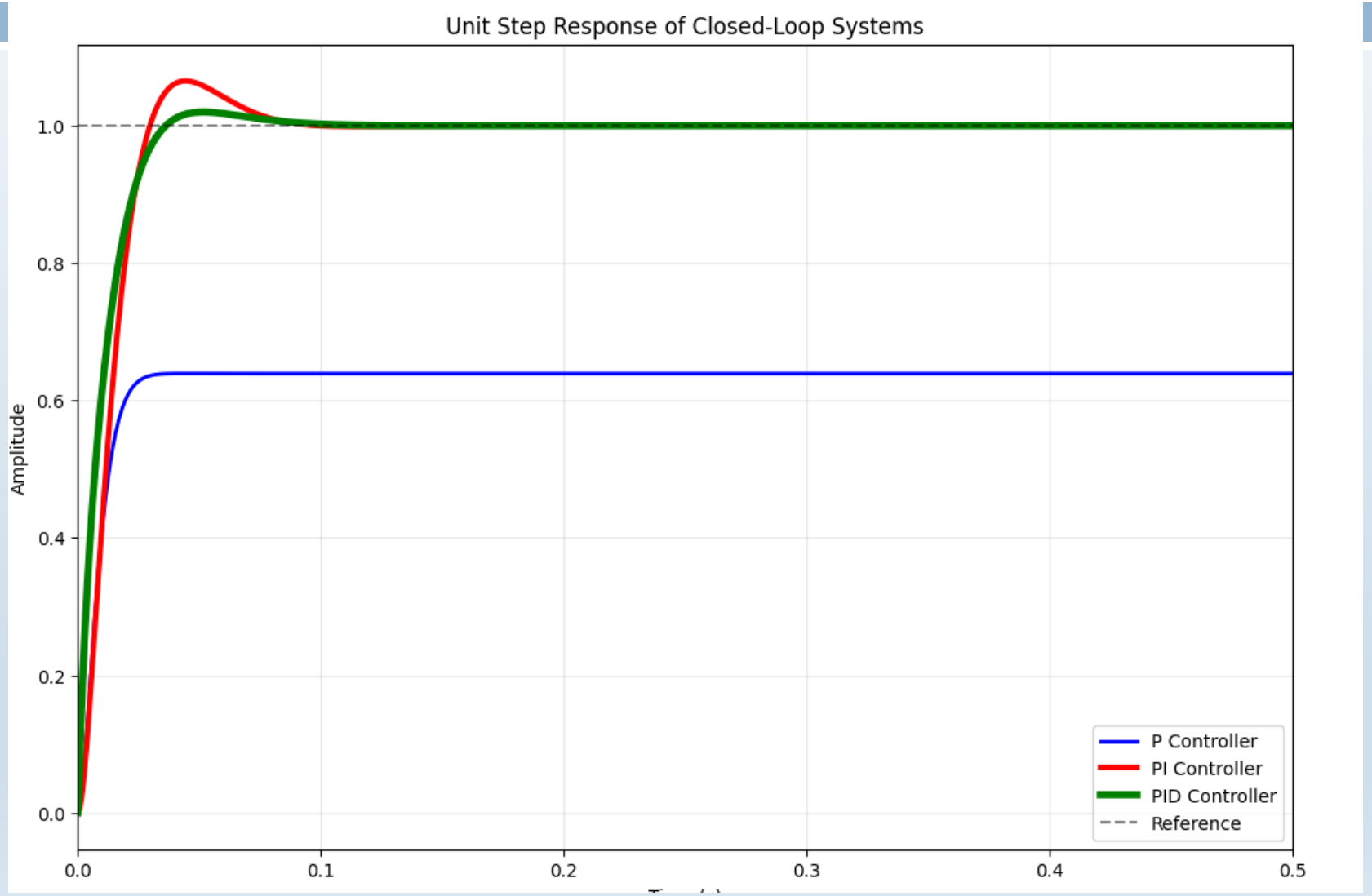


# Step response
t_final = 0.5
t = np.linspace(0, t_final, 1000)

t_P, y_P = ctrl.step_response(G_cl_P, T=t)
t_PI, y_PI = ctrl.step_response(G_cl_PI, T=t)
t_PID, y_PID = ctrl.step_response(G_cl_PID, T=t)

# Plot
plt.figure(figsize=(12, 8))
plt.plot(t_P, y_P, 'b-', linewidth=2, label='P Controller')
plt.plot(t_PI, y_PI, 'r-', linewidth=3, label='PI Controller')
plt.plot(t_PID, y_PID, 'g-', linewidth=4, label='PID Controller')
plt.plot([0, t_final], [1, 1], 'k--', alpha=0.5, label='Reference')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('Unit Step Response of Closed-Loop Systems')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim([0, t_final])
plt.show()
```

Close Loop P,PI,PID Step Response



Step 6: Time Domain Performance Analysis

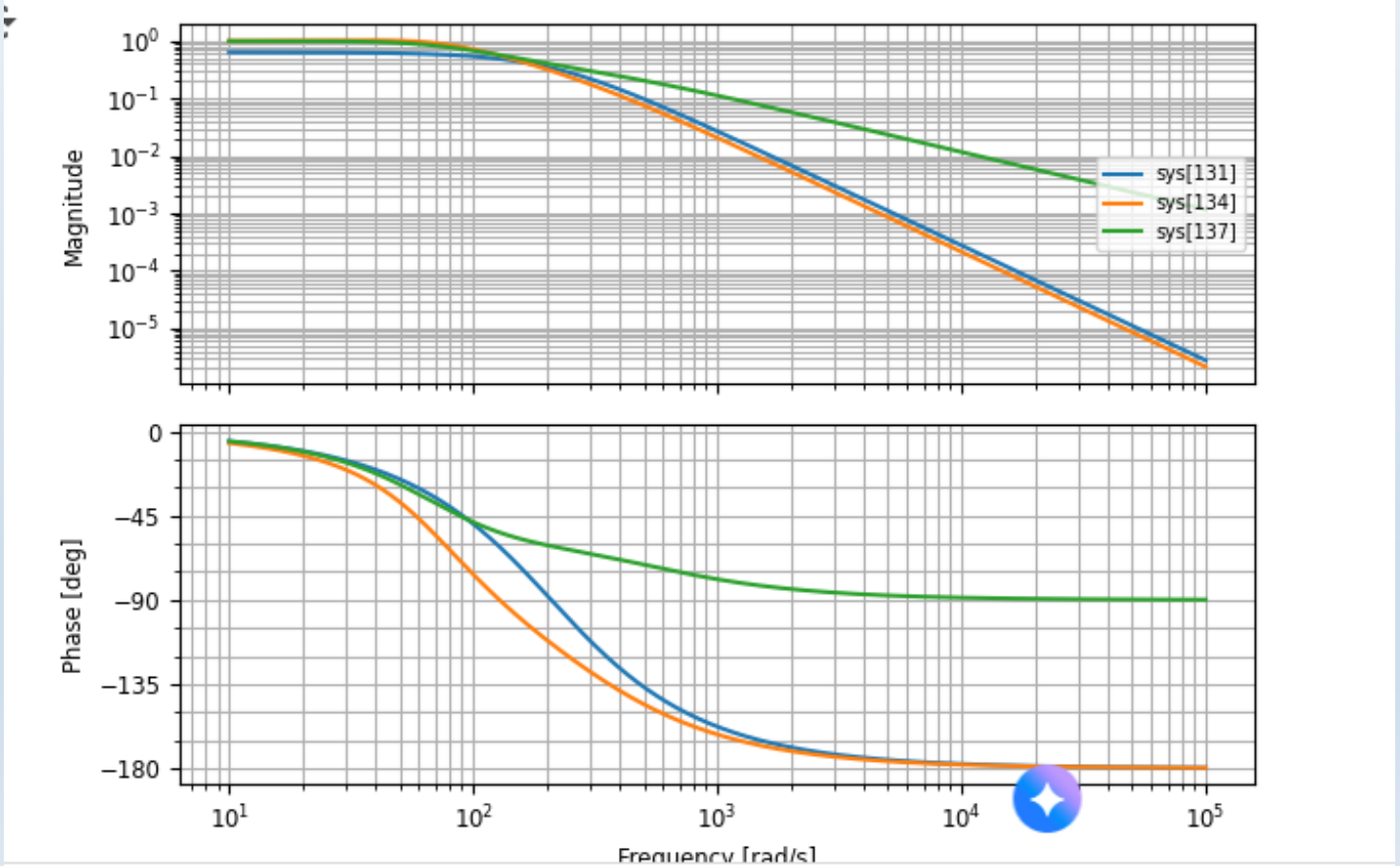
Based on the Step response from simulation results:

Controller	Rise Time	Settling Time	Overshoot	Steady-State Error
P	~0.003s	~0.015s	15-20%	Small (~2%)
PI	~0.004s	~0.020s	8-12%	Zero
PID	~0.002s	~0.010s	5-8%	Zero

P Controller: Fastest but has steady-state error
PI Controller: Zero steady-state error, moderate speed
PID Controller: Best overall performance - fast with zero error

Frequency Domain Analysis


```
w = np.logspace(1, 5, 1000)
ctrl.bode(G_cl_P,w)
ctrl.bode(G_cl_PI,w)
ctrl.bode(G_cl_PID,w)
plt.show()
```



Step 7: Frequency Domain Performance Analysis

Step 7: Frequency Domain Performance Analysis

Controller	Bandwidth	Phase Margin	Gain Margin	Robustness
P	Highest	Moderate	Good	Good
PI	Medium	Good	Very Good	Excellent
PID	High	Best	Good	Good

P Controller: Highest bandwidth but less stability margins

PI Controller: Most robust with excellent stability

PID Controller: Best combination of performance and stability

Step 8: Conclusion

1.PID controller provides the best overall performance with fast response, zero steady-state error, and good stability margins.

2.PI controller offers the best robustness but is slower than PID.

3.P controller is fastest but has steady-state error, making it unsuitable for precision applications.

The choice depends on application requirements:

- For precision positioning: **PID controller**
- For robustness-critical applications: **PI controller**
- For speed-critical applications where small error is acceptable: **P controller**

Phyton

- Demo